

AI-Suplex 3-Layer Memory Stack

AI-Suplex 3-Layer Memory Stack

TWABAM ⚡! The memory layer turns AI-Suplex from a one-off workflow helper into a system that remembers, reuses, and improves. Every Tasklist execution makes the next one smarter.

What It Does

The memory stack preserves context across sessions, converts experience into durable knowledge, and keeps the vault organized around real execution. Three layers, one purpose: compounding usefulness over time.

Layer	Location	What It Stores
Episodic	memory/episodic/Cycle-X/Week-Y/	What happened — session records, outcomes, scores
Semantic	memory/semantic/	Stable truths — preferences, rules, glossary, entities
Procedural	memory/procedural/	Reusable methods — workflows, macros, prompt patterns
Lessons	memory/lessons.md	Candidate learnings — staging area before promotion

The Self-Improving Loop

```
Session End Report → 3lm end (write episode)
→ 3lm learn (extract lessons)
→ 3lm promote --min 70 (score + promote)
→ 3lm revise (check contradictions)
→ 3lm index (refresh lookup)
→ Next session starts smarter ✂
```

The 3lm CLI Tool

The `3lm` tool manages vault operations. Run from inside `AI-Suplex-777/` :

```
node scripts/3lm/3lm.js <command>
```

Command	What It Does
<code>start</code>	Load all memory context — semantic, lessons, episodes, procedures
<code>end</code>	Write episodic file from your session report
<code>learn</code>	Extract lessons from latest episode
<code>promote --min 70</code>	Score lessons, promote high-scoring ones to semantic/procedural
<code>revise</code>	Check for contradictions, surface deprecation watch
<code>index</code>	Refresh the memory lookup manifest
<code>status</code>	Show counts across all memory stores

After every session:

```
node scripts/3lm/3lm.js end      # Write the episode
node scripts/3lm/3lm.js learn    # Extract lessons
node scripts/3lm/3lm.js index    # Refresh index
```

Weekly review:

```
node scripts/3lm/3lm.js promote --min 70 # Promote stable lessons
node scripts/3lm/3lm.js revise           # Check for contradictions
```

CortexMem: Optional Bridge (Optimizer)

CortexMem is a **fast-capture bridge** for selected high-value context. It is **not required** — the markdown vault is the canonical memory store. The 3-layer memory stack works without it.

Use CortexMem only for:

- Major decisions you want to capture quickly
- Explicit constraints and preferences
- Compact session summaries
- Bridge-state continuity between tools

If you choose to set up CortexMem:

```
npm install -g cortexmem
cd "path/to/AI-Suplex-777"
```

```
git init
node cm get_status
```

CortexMem commands (`node cm save_context` , `node cm get_context`) remain available as an optional acceleration layer. When CortexMem and the vault disagree, the **reviewed markdown vault wins**.

Promotion Rules

Path	Threshold
Episodic → Semantic	Fact appears in 3+ sessions and has not changed
Episodic → Procedural	Workflow succeeds 3 times with similar steps
Lessons → Procedural	Candidate scores 70+ on the 100-point rubric
Keep in lessons	Useful but not yet stable
Deprecate	Superseded by newer, stronger evidence

Memory Folder Structure

```
memory/
├─ memory-lifecycle.md      # How memory evolves over time
├─ promotion-rules.md      # What gets promoted and when
├─ contradiction-rules.md  # How conflicts are handled
├─ review-cadence.md       # When reviews happen
├─ index.md                # Quick lookup map
├─ lessons.md              # Staging area
├─ semantic/
│   ├─ preferences.md      # How we work
│   ├─ project_rules.md    # What governs the system
│   ├─ glossary.md         # What terms mean
│   └─ entities.md         # Who and what exists
├─ procedural/
│   ├─ tasklist_generation.md # Raw input → Tasklist
│   ├─ session_start.md     # Tasklist → Execution
│   ├─ session_end.md       # Execution → Report
│   ├─ weekly_review.md     # Week → Patterns
│   ├─ cycle_review.md      # Cycle → Consolidation
│   ├─ promotion_rules.md   # Lessons → Memory
│   └─ lesson_to_procedure.md # Learning → Automation
└─ episodic/
```

└─ Cycle-1/
└─ Week-1/

Your session episodes go here

North Star

"Each new session starts smarter than the last."

Every Tasklist execution should leave the system better than it found it. The memory stack makes this automatic — capture what happened, promote what mattered, reuse what worked.

Memory is live. Each session feeds the engine.  